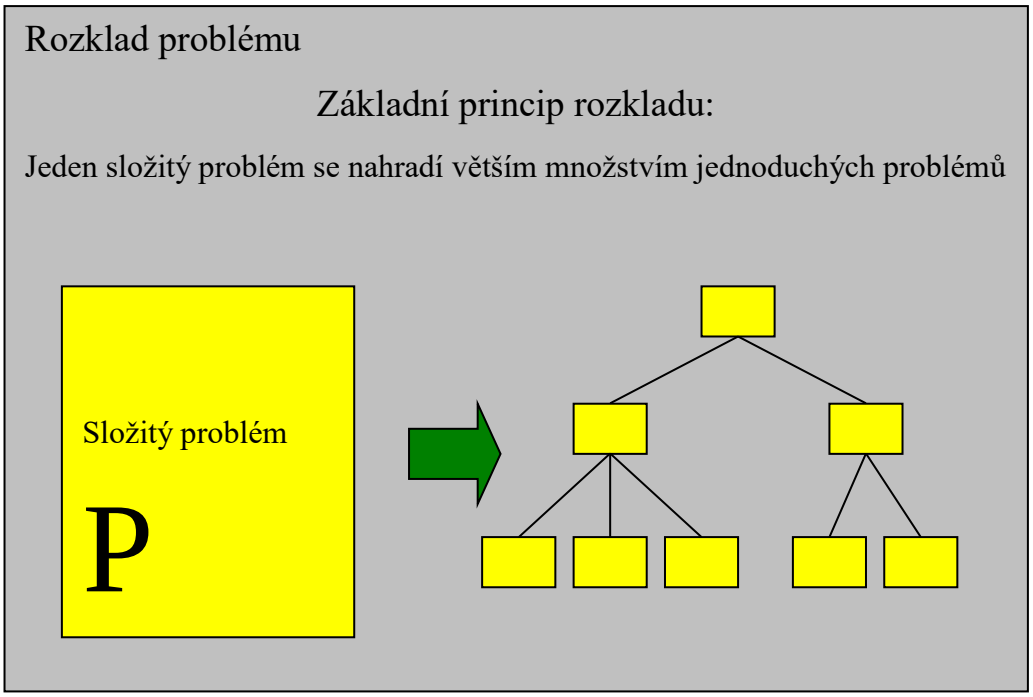
Principy strukturovaného programování

Strukturovaného programování se na konci 60. let objevilo jako nový fenomén, který měl zbavit programování jeho dětských nemocí, které vyústily v krizi softwaru. Jednalo se o první výraznou koncepci programování, která znamenala nejen změnu v přístupu k vývoji aplikací, ale stala se významným podnětem i pro vznik zcela nových strukturovaných jazyků, jejichž mutace se v programování prosazují i v současnosti – Pascalu a jazyka C. Program začal být důsledně chápán jako přehledná logická struktura, v níž se větší celky skládají z dílčích částí a kde se mnohem více dbá na rozlišování datových typů.

Proč strukturovaný přístup?

Našim cílem je ***vytvářet programy bez chyb***. Jak zajistíme, aby byl program bezchybný? Například jeho testováním, které zaručí minimálně to, že testovaný případ proběhl bez chyby. Ostatní podobné případy budou s velkou pravděpodobností také dobře, ale jen pravděpodobně, nikdy ne jistě. Program by měl být tak jednoduše napsán, aby bylo možné snadno pochopit jeho činnost a již z jeho struktury poznat, zdali neobsahuje chyby. Proto je základním principem strukturovaného přístupu ***rozklad*** (dekompozice), kdy jeden složitý problém je nahrazen větším množstvím jednodušších problémů.



Základní řídicí struktury a jejich vyjádření

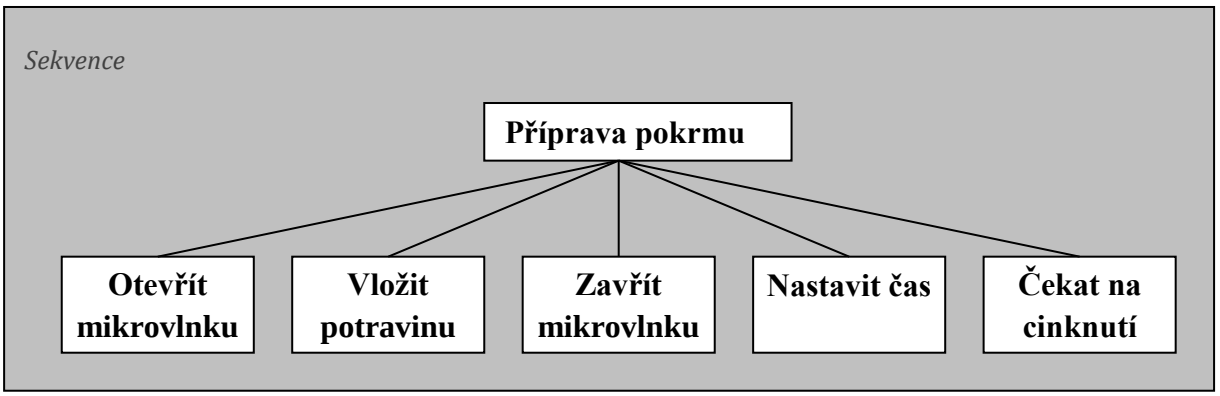
Aby mohl být program snadno pochopitelný, používají se jen ty nejjednodušší struktury. Je matematicky dokázáno, že pomocí těchto řídicích struktur lze vyjádřit libovolně složitý algoritmus.

Sekvence

Je posloupnost činností nebo prvků. Z teoretického hlediska to lze brát tak, že skupinu pojmů, které mají mezi sebou určitou vazbu (následují bezprostředně za sebou), označíme jedním abstraktnějším pojmem. Tak například posloupnost po sobě následujících jídel označíme jako oběd.

Sekvence můžeme vytvářet ve všech oblastech. Lze tvořit sekvenci hmotných předmětů (oběd je posloupnost jídel), abstraktních informací (záznam o zaměstnanci je sekvence informací o vzdělání, odbornosti a současného postavení), ale také činností s hmotnými předměty a operacemi s informacemi nebo daty.

Jestliže mluvíme o tvorbě sekvencí, tak se vždy pohybujeme na určité úrovni podrobností. Tak například "Přípravu pokrmu" označíme jako sekvenci otevření mikrovlnky, vložení potraviny a tak dále; zanedbáváme přitom to, že otevření mikrovlnky samo o sobě je dosti složitá činnost.



Selekce

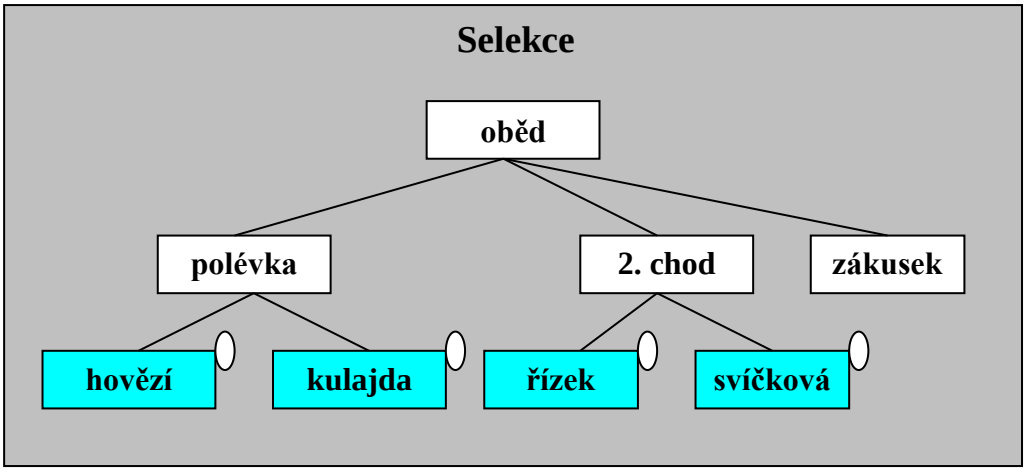
Výběr z možností. Například k obědu si mohu vybrat jen to, co je na jídelním lístku. Zase vidíme, že selekci lze použít v nejrůznějších oblastech (jako to bylo u sekvence). Je velmi důležité, aby podmínky byly výlučné a úplné.

Výlučnost podmínek znamená, že v selekci smí současně platit jen jedna podmínka, podmínky se nesmí překrývat.

Úplnost podmínek znamená, že v selekci musí platit alespoň jedna podmínka, tj. že alespoň jedna možnost musí být vybrána.

V jazyce C mohou být selekce vyjádřeny příkazy ***if*** a ***case***.

Podmínka je v následujícím strukturogramu vyjádřena malou elipsou v pravém horním rohu okénka.

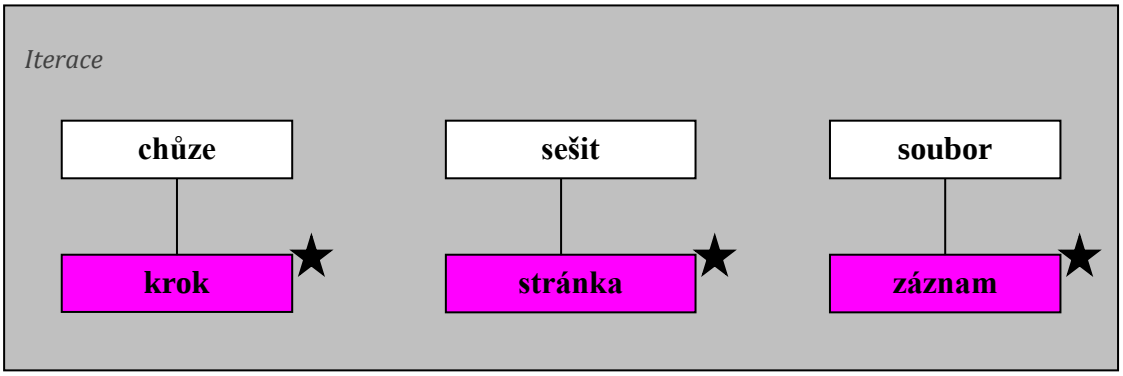


Iterace

Opakování. Znáte mnoho příkladů z běžné praxe – chůze je iterací kroků, kilometr je iterace metrů, sešit je iterace jednotlivých listů apod. Programátory samozřejmě nejvíce zajímají iterace operací s daty. Iterace je opakování části operací (příkazů).

V jazyce C mohou být iterace vyjádřeny příkazy ***for***, ***do..while***, ***while***.

Na následujícím obrázku jsou uvedeny příklady iterací. Hvězdička představuje podmínku, která musí být splněna, aby bylo opakování prováděno. Pokud splněna není, opakování bude ukončeno.



Úvodem k příkazům a k řízení běhu programu

Základním stavebním prvkem zdrojového kódu je příkaz. Vytváříme-li program, zapisujeme vlastně posloupnost příkazů. Tato posloupnost bude za běhu programu vykonávána.

Příkazy specifikují „tok výpočtu“, neboli „běh programu“. Provádějí se postupně v pořadí, v jakém jsou zapsány. Přemýšlíme-li o tom, jak naprogramovat nějaký problém, je nutné vymyslet vhodnou posloupnost příkazů (a volání funkcí), která povede k požadovanému řešení. Vhodnou kombinací příkazů a funkcí je možné naprogramovat v podstatě cokoliv.

Prázdný příkaz

Prázdný příkaz je tvořen samostatným středníkem umístěným ve zdrojovém kódu.

Blok

Zatímco prázdný příkaz se používá relativně zřídka, blok nacházíme ve zdrojovém textu kódu velmi často. Blok je skupina příkazů uzavřená ve {složených závorkách}. Ve zdrojovém kódu může být uveden tam, kde se očekává příkaz jediný. Tedy na místo, kam nám syntaxe jazyka umožňuje uvést jen jeden příkaz, můžeme stejně tak uvést blok příkazů. Blok příkazů „navenek“ vystupuje jako příkaz jediný.

Příkazy uvnitř bloku se také provádějí postupně. Kromě příkazů (a samozřejmě také volání funkcí apod.) může blok obsahovat i definice proměnných – a, nebo třeba další (vnořený) blok.

Otvírací složená závorka bloku by měla být na samostatné řádce, všechny příkazy bloku by pak měly být odsazeny určitý počet znaků od levého okraje – počet odsazených znaků záleží na každém programátorovi, často se uvádí jako optimální počet dva znaky.

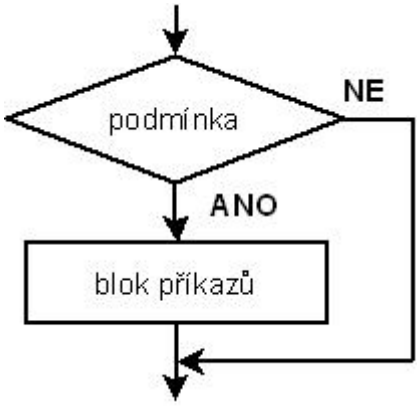
```
{                          /* složená závorka označuje počátek bloku */
  int i = 0, j = 3;        /* definice proměnné s inicializací */
  int k;                   /* definice proměnné */

  k = (i && j) || (i == 5);          /* příkaz */
  printf(„%d %d %d\n“, i, j, k);     /* volání funkce printf() */

  scanf(„%d“, &j);                   /* volání funkce scanf() */
}                                    /* konec bloku */
```

Příkaz if a if-else

Příkaz `if` (a jeho „rozšíření“ `if-else`) se nazývá podmíněný příkaz. Slouží k rozhodnutí o dalším provádění programu v závislosti na hodnotě logického výrazu (podmínky).

```
if (logický_výraz)
      příkaz_1;
else
      příkaz_2;
```

Vývojový diagram



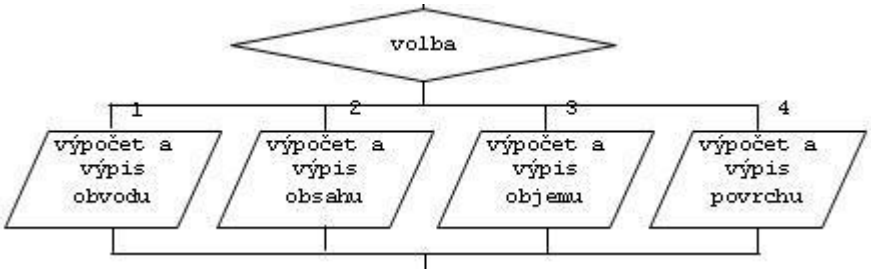
- Celá větev `else příkaz_2` je **nepovinná** a nemusí se v příkazu vůbec vyskytnout.
- Na místě kteréhokoliv příkazu (ať již `příkaz_1, příkaz_2` nebo jakýkoliv příkaz uvnitř případného bloku) je možné uvést další příkaz `if` nebo `if-else`. Je tedy zřejmé, že příkazy `if` se mohou vnořovat.
- V souvislosti s příkazem `if-else` je nezbytně nutná znalost logických operátorů, logických výrazů a způsobu jejich vyhodnocování.

Přepínač switch

Příkaz if-else má jednu nevýhodu: nedokáže postihnout více než dvě varianty: pokud výraz platí, proveď něco, pokud neplatí, proveď něco jiného.. V případě vícenásobného větvení použijeme příkaz switch:

```
switch (výraz) {
  case konstanta_1 :
       příkaz_1;
       break;
  case konstanta_2 :
       příkaz_2;
       break;
  ...
  case konstanta_N :
       příkaz_N;
       break;
  default :
      příkaz_def;
      break;
}
```

Vývojový diagram



Přepínač funguje jako speciální vícecestná rozhodovací konstrukce. Umožňuje provést příkaz v závislosti na hodnotě výrazu. V tomto případě se nebude rozhodovat na základě logického výrazu ale na základě celočíselného. Ten může samozřejmě nabývat mnohem více hodnot.

Způsob funkce přepínače: pokud je hodnota výrazu rovna `konstanta_1`, provede se `příkaz_1`. Pokud je hodnota výrazu rovna `konstanta_2`, provede se `příkaz_2`, a tak dále. Neodpovídá-li hodnota výrazu žádné z konstant, provede se `příkaz_def`.

Příkaz `switch` umožňuje celou řadu modifikací. Uvedení příkazu break není povinné. Pokud na konci některé z větví chybí, program neopouští příkaz switch a pokračuje vykonáváním příkazů následující větve v pořadí.

- Větev `default` nemusí být uvedena jako poslední v pořadí, nicméně i když bude zapsána třeba jako první, stejně se nejprve zkontrolují ostatní větve a teprve pokud není nalezena žádná shoda, provede se větev default. Také na pořadí větví `case` nezáleží.
- Větev `default` nemusí být uvedena vůbec. Pokud chybí a žádná větev „nezabere“, příkaz `switch` neprovede nic a vykonávání programu pokračuje příkazem následujícím po konci `switch`.
- Konstanty, podle kterých se rozhoduje, musí být buď celočíselné, nebo znakové.
- Za poslední větví nemusí být uvedeno klíčové slovo `break`. Jeho uvedení však svědčí o dobrých programátorských návycích a kromě toho usnadní budoucí úpravy zdrojového kódu.

Příkazy pro vytváření cyklů

Příkazy pro vytváření cyklů používáme vždy, když potřebujeme nějakou činnost (akci) provádět opakovaně.

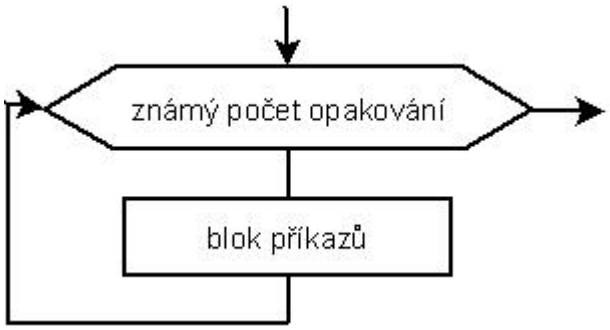
Jazyk *C* obsahuje tři základní typy cyklů: cyklus `for`, cyklus `while` a cyklus `do while`.

Cyklus for

Cyklus for bývá také nazýván **cyklus se známým počtem průchodů.** Používáme jej v případě, že už před jeho prvním průchodem víme, kolikrát se bude muset provést. Pokud je třeba ve třídě určitý počet žáků a pro každého z nich potřebujeme vypsat na obrazovku nějaký údaj, použijeme cyklus for. Syntaxe je následující:

```
for (počáteční_výraz; koncový_výraz; iterační_výraz)
  příkaz;
```

Vývojový diagram



Tři výrazy uvedené v závorce a oddělené středníky se nazývají **záhlaví cyklu**, příkaz tvoří **tělo cyklu.** Na úplném začátku, ještě před prvním průchodem, se vyhodnotí výraz `počáteční_výraz`. Toho se často používá např. pro inicializaci (vynulování) proměnné, která nese informaci o počtu provedených průchodů. Vzápětí se vyhodnotí také výraz `koncový_výraz`. Pokud má hodnotu 0 (je nepravdivý), cyklus skončí (vlastně ještě dříve než začal), neprovede se ani jednou a provádění programu pokračuje za cyklem. Je-li však `koncový_výraz` pravdivý, provede se `příkaz`. Nakonec se vyhodnotí `iterační_výraz`, tím končí jedna obrátka cyklu a začíná další. V dalších obrátkách se nebude vyhodnocovat `počáteční_výraz`, další iterace budou spočívat jen ve vyhodnocení výrazu `koncový_výraz`, v provedení příkazu a ve vyhodnocení výrazu `iterační_výraz`. Provádění cyklu končí v okamžiku, kdy `koncový_výraz` bude mít nepravdivou hodnotu.

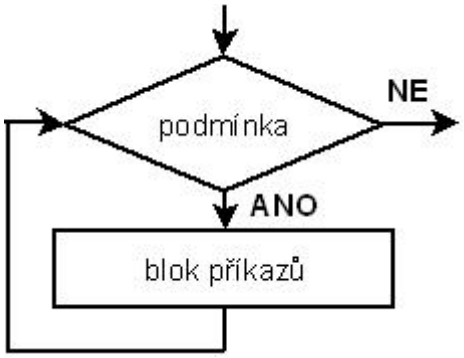
Ani jeden ze tří výrazů příkazu `for` (počáteční, koncový, iterační) není povinný a nemusí být uveden. Například počáteční výraz si můžeme vyhodnotit už dávno před zavoláním příkazu `for` apod. V každém případě však musíme zapsat oba dva středníky, které výrazy oddělují.

Cyklus while

Cyklus while bývá také nazýván **cyklus s podmínkou na začátku.** Používá se v případech, kdy dopředu neznáme počet iterací (nelze tedy použít cyklus for) a kdy se také může stát, že **cyklus nebude muset proběhnout ani jednou.** Syntaxe:

```
while (výraz)
  příkaz;
```

Vývojový diagram



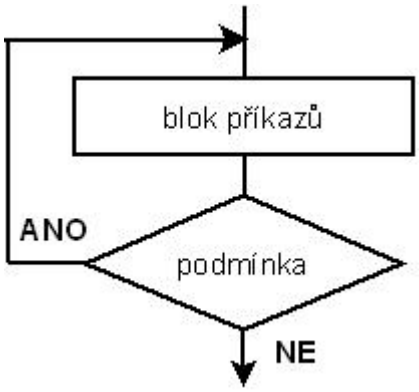
Popis: Výraz (podmínka), podle kterého se rozhoduje o provádění/neprovádění iterace cyklů, se testuje ještě před prvním průchodem (podobně jako u cyklu for). Iterace (klidně i první) se nezačne provádět, pokud je tento výraz nepravdivý. Výraz se opět nazývá záhlaví cyklu, příkaz je jeho tělem. Výraz musí být uveden v závorkách.

Cyklus do while

Posledním cyklem je cyklus, který bývá též nazýván **cyklus s podmínkou na konci.** Používá se v případech, kdy víme, že **bude potřeba alespoň jeden průběh,** ale přesný počet průběhů dopředu neznáme. Syntaxe:

```
do {
  příkazy
} while (výraz);
```

Vývojový diagram



I v tomto cyklu se tedy vyskytuje klíčové slovo `while`. Na rozdíl od cyklu `while` (popsaného v předchozí podkapitole) se však uvádí až na konci – výraz uvedený za ním se bude vyhodnocovat až po skončení iterace. První iterace se tedy provede vždy, bez ohledu na hodnotu výrazu. Další iterace se provede pouze v případě, že výraz je pravdivý. Výraz musí být opět uveden v závorkách.

Příkazy break a continue

Na závěr pojednání o cyklech jsme si nechali příkazy break a continue. Tyto příkazy se totiž týkají všech cyklů: for, while i do. Můžeme je použít v těle kteréhokoliv z nich a jejich zavolání se okamžitě projeví na dalším způsobu provádění cyklu:

- Příkaz `break` způsobí okamžité vyskočení z cyklu. **Cyklus je tedy ukončen** a provádění programu pokračuje následujícím příkazem za cyklem.
- Příkaz `continue` způsobí okamžité ukončení aktuální iterace a vynutí bezprostřední započetí další iterace. Přesněji řečeno – `continue` způsobí vyhodnocení podmínky cyklu, takže je-li tento výraz pravdivý, provádění programu pokračuje další iterací cyklu. **Cyklus tedy není ukončen.**

Funkce

Jednou ze zásad strukturovaného programování je **hierarchický rozklad**, *což znamená, že jeden složitý problém se nahradí větším množstvím jednoduchých problémů. V programátorské praxi je tato zásada mimo jiné realizována členěním rozsáhlejších programů na dílčí části (***funkce***), případně vytváření oddělených programových modulů.*

Co jsou funkce

Každý program v jazyce *C* se skládá z jedné nebo více funkcí. Hierarchie funkcí vytváří určitou strukturu, kterou provádění programu postupně prochází. **Funkce je základním prostředkem strukturování programu.** Jazyk *C* je nazýván strukturovaným jazykem.

Funkce umožňují členit rozsáhlé úlohy na menší celky a také stavět na tom, co už bylo jednou naprogramováno. Existuje ohromné množství funkcí, které jsou k dispozici v **knihovnách** (říkáme jim knihovní funkce). Knihovna je (zjednodušeně řečeno) soubor obsahující několik (řadu) funkcí, které můžeme každý přímo využít ve svém programu. Používání knihovních funkcí je mimořádně jednoduché – stačí mít knihovnu (s jazykem *C* se jich dodává dlouhá řada) a zpřístupnit tuto knihovnu ve svém programu direktivou `#include`.

Kromě používání knihovních funkcí však při programování vytváříme své vlastní. Vždy je třeba mít na paměti, že funkce je samostatná jednotka, která by měla provádět jednu specializovanou činnost. Můžeme například vytvořit funkci pro výpočet faktoriálu, pro výpočet mocniny čísla, ale také pro získání uživatelského vstupu, pro výpis chybového hlášení apod. Když se chystáme vytvořit program, nejprve si vše důkladně rozmyslíme – a výsledkem těchto úvah by mělo být „rozkouskování“ problému na dílčí, relativně samostatné „podproblémy“ (odborně to nazýváme **dekompozice problému**). Programování pak vypadá tak, že pro každý z těchto problémů vytvoříme vlastní funkci a v hlavním programu (tedy ve funkci `main()`) ve vhodném pořadí tyto funkce zavoláme.

Je samozřejmé, že dělení problému na podproblémy může mít více „vrstev“ – například hlavní funkce `main()` zavolá funkci `načti_soubor()`, a ta si postupně zavolá funkce `ověř_existenci_souboru()`, `otevři_soubor()`, `čti_soubor()` a `uzavři_soubor()`. Ani zde ale nemusí funkční hierarchie končit, například funkce `čti_soubor()` může v cyklu volat funkci `čti_řádku_souboru()` apod.

Identifikátor funkce, deklarace a definice

Identifikátor funkce je vlastně jejím pojmenováním.

Velmi důležité jsou dva následující pojmy: deklarace a definice.

Deklarací pouze přiřazujeme funkci její identifikátor. V rámci deklarace se nepřiděluje žádná paměť a neprovádí žádná činnost, pouze se uvede její **hlavička** (tj. jméno funkce, její návratový typ a parametry).

Definice je náročnější: proměnné přidělí jméno i paměťový prostor, funkci přidělí jméno a zároveň obsahuje celé tělo funkce (všechny její příkazy). Deklarujeme-li funkci, uvádíme pouze její název (a případné parametry), žádné příkazy (žádné tělo funkce). Definice funkce naopak znamená kompletní uvedení funkce – tedy název, případné parametry a celé tělo (všechny příkazy).

Hlavička funkce ukončená středníkem bývá také nazývána **úplný funkční prototyp**.

```
int mocnina(int, int );
```

Návratová hodnota funkce a příkaz return

Každá funkce v jazyce C musí vracet nějaký údaj. Datový typ tohoto údaje se stává **návratovým typem funkce,** takže funkce vracející celé číslu bude mít celočíselný návratový typ. Uvádění návratového typu je součástí deklarace a definice funkce. Pokud programujeme funkce, z níž skutečně nepotřebujeme nic vrátit, můžeme jako návratový typ uvést `void`.

Příkaz `return` se nemusí vyskytovat až na konci těla funkce, může být uveden kdekoliv – způsobí však okamžité ukončení běhu funkce a předání řízení volající funkci. Použití příkazu `return` ve funkci `main()` způsobí ukončení celého programu.

Příkaz `return` se v těle funkce také nemusí objevit jen jednou. Používáme-li ve funkci nějaký příkaz větvení (`if-else`, přepínač apod.), je možné použít `return` v každé z větví.

Hlavičkové soubory

Při programování v jazyce C je důležitou součástí používání hlavičkových souborů. Hlavičkové soubory obsahují hlavičky funkcí, přesněji řečeno úplné funkční prototypy (návratový typ funkce, identifikátor a seznam datových typů formálních parametrů).

Hlavičkové soubory mají příponu `.h` (v C+ `.hpp`)a označují se také *header-files* či *headers*. Klíčovou roli hrají při správě rozsáhlejších projektů.

Hlavičkové soubory si můžeme vytvářet buď sami při vytváření nových uživatelských knihoven, pak jim říkáme **uživatelské hlavičkové soubory** nebo můžeme využít již hotových **standardních hlavičkových souborů**.

Hlavičkový soubor vložíme do zdrojového souboru direktivou preprocesoru `#include`. Vložením dáme překladači k dispozici úplné funkční prototypy funkcí definovaných v jiném modulu, takže je ve svém zdrojovém textu můžeme bez obav volat.

Vkládání hlavičkových souborů se provádí dvojím způsobem. Názvy uživatelských hlavičkových souborů zapisujeme v uvozovkách. Názvy standardních hlavičkových souborů uvádíme ve špičatých závorkách.

```
#include <stdio.h>

#include "muj_hlavickovy_soubor.h"
```